

Università degli Studi di Napoli Federico II

Corso di Programmazione Object Oriented

Documentazione finale del progetto
gestione ospedaliera

Gruppo 03

- Daniele Di Napoli DE1000177
- Arianna Iovine DE1000122

Docente:

Prof. Porfirio Tramontana
Anno accademico 2025/2026

La presente documentazione descrive l'architettura, le tecnologie utilizzate, la struttura dei package, il collegamento con il database PostgreSQL e i principali diagrammi UML relativi al progetto sviluppato.

INDICE

Introduzione.....	1
Tecnologie utilizzate.....	2
Architettura del sistema.....	2
Descrizione dei package.....	3
Database PostgreSQL.....	3
Esempio di vulnerabilità.....	4
Diagrammi UML.....	4
Sequence Diagram.....	7
Conclusioni.....	7

Introduzione

Il progetto da noi realizzato consiste nello sviluppo di un'applicazione Java per la gestione di alcune funzionalità di base di una struttura ospedaliera. L'obiettivo principale di quest'ultimo è quello di permettere ad un amministratore di sistema di accedere all'applicazione e gestire l'archivio dei pazienti, aggiungere nuove degenze e consultare la dashboard principale.

Il lavoro è stato realizzato articolando il sistema in package, per essere specifico utilizzando una struttura architetturale BCE (Boundary, Controller, Entity). Il package boundary è rappresentato dal package gui, il quale contiene classi java per la rappresentazione grafica. Il package controller invece si occupa di coordinare tutte le operazioni logico-matematiche. Il package entity contiene tutte le classi del dominio (Paziente, Ricovero, Letto...).

La presente documentazione del progetto include i principali diagrammi UML richiesti dalla consegna dal prof. Porfirio Tramontana. Sono stati realizzati diversi diagrammi relativi ai package principali del sistema, tra cui package diagrams e 2 sequence diagrams.

Il progetto pertanto risponde agli obiettivi formativi principali dell'homework tra cui:

- Collegamento del database al progetto Java.
- Documentazione del codice e utilizzo di Javadoc.
- Utilizzo di PostgreSQL, JDBC, Swing...
- Implementazione di tutti i packages (database, dao, implementazioneDAO...).
- Utilizzo di un sistema di versioning (github).
- Lavoro di squadra

Tecnologie utilizzate

Il progetto è stato sviluppato utilizzando Java, il linguaggio principale con cui è stata scritta la logica dell'applicazione. L'interfaccia grafica, invece, è stata realizzata con la libreria Java Swing, che ci ha consentito di creare le diverse finestre, i campi di testo e i bottoni necessari per far interagire l'utente con il sistema. Tutti i dati, ad esempio quelli dei pazienti e dei ricoveri, sono memorizzati su un database "relazionale" (cioè composto da righe e colonne) PostgreSQL. Per far interagire il nostro codice Java con il database abbiamo utilizzato JDBC (Java Database Connectivity), tramite questo strumento siamo stati in grado di inviare direttamente dal programma le query SQL per leggere, inserire o modificare i dati (CRUD operations)

L'IDE su cui abbiamo lavorato al progetto è IntelliJ IDEA, mentre per effettuare i commit e il push del progetto finale abbiamo utilizzato Git e GitHub.

Architettura del sistema

In questo progetto è stato adottato il modello architetturale Boundary-Control-Entity (BCE, talvolta indicato anche come EBC). Si tratta di un modello architetturale usata nei casi d'uso nella programmazione orientata agli oggetti. Questo sistema si basa su tre responsabilità:

- **Boundary:** Rappresenta la schermata GUI del progetto
- **Controller:** Gestisce le operazioni logiche-matematiche.
- **Entity:** Rappresenta le entità del progetto

Questi tre componenti collaborano tra di loro per garantire un sistema efficiente, infatti il Boundary comunica esclusivamente con il Controller. Il Controller, a sua volta, interagisce con le Entity.

Descrizione dei package

Il progetto da noi costruito è organizzato in diversi package (cartelle), ognuno dei quali ha una responsabilità specifica (fare riferimento all'architettura di sistema).

Questa suddivisione permette di mantenere il codice più ordinato e di separare le varie parti dell'applicazione: interfaccia grafica, logica di controllo, modello del dominio, accesso ai dati e connessione al database, ognuna gestita da un package differente.

I packages utilizzati in questo progetto sono:

- **Gui**: gestisce la parte grafica del progetto.
- **Controller**: gestisce le operazioni logiche-matematiche.
- **Model**: contiene le classi principali e rappresentano le entità del dominio.
- **DAO**: contiene le interfacce DAO, cioè particolari interfacce che definiscono le operazioni disponibili per l'accesso ai dati.
- **ImplementazioneDAO**: contiene le classi concrete che implementano le interfacce definite dal package DAO
- **database_connection**: gestisce la connessione al database PostgreSQL

Database PostgreSQL

Per la gestione dei dati è stato utilizzato un database, cioè uno strumento di **collezione dati persistente**. Nel nostro progetto abbiamo utilizzato PostgreSQL, un sistema di gestione di DBMS. Come già descritto precedentemente, è stato utilizzato un sistema di JDBC (Java DataBase Connectivity) per effettuare il collegamento tra Java e SQL ed effettuare le query.

Nel database sono state definite le tabelle principali tra cui:

- utente
- paziente
- reparto
- stanza
- letto
- ricovero

Durante lo sviluppo del database abbiamo avuto modo di sfruttare le conoscenze acquisite del corso di "Basi di Dati" del professore Silvio Barra.

Quando un applicativo interagisce con un database, c'è sempre il rischio che alcuni hacker possano sfruttare una vulnerabilità chiamata "SQL Injection", cioè un tipo di attacco in un cui il malintenzionato, approfittando di una scarsa gestione dell'input, inietta codice *malevolo* all'interno del database.

Esempio di vulnerabilità

Prendiamo ad esempio il seguente codice:

```
String query_da_inviare = "SELECT * FROM users WHERE name = '"  
+ query_da_inviare + "'";
```

Questo codice SQL estrae il record del nome utente specificato dalla sua tabella degli utenti. Il problema è che se la variabile "query_da_inviare" è manipolata in modo specifico, l'istruzione SQL può fare più di quanto previsto dal programmatore

Ad esempio, impostando la variabile "query_da_inviare" come:

```
' OR '1'='1 --
```

Il codice diventa: `SELECT * FROM users WHERE name = ' OR '1'='1 --;`

Se questo codice venisse utilizzato in una procedura di autenticazione, il database potrebbe restituire utenti anche senza inserire le credenziali corrette, in quanto la condizione non controlla più solo il login specifico inserito dall'utente viene "forzata" a risultare vera.

Oppure potrebbe essere utilizzata per rimuovere tutte le tabelle.

Nel nostro progetto facciamo largo uso di [PreparedStatements](#), cioè strumenti che ci aiutano a ridurre il rischio di SQL Injection.

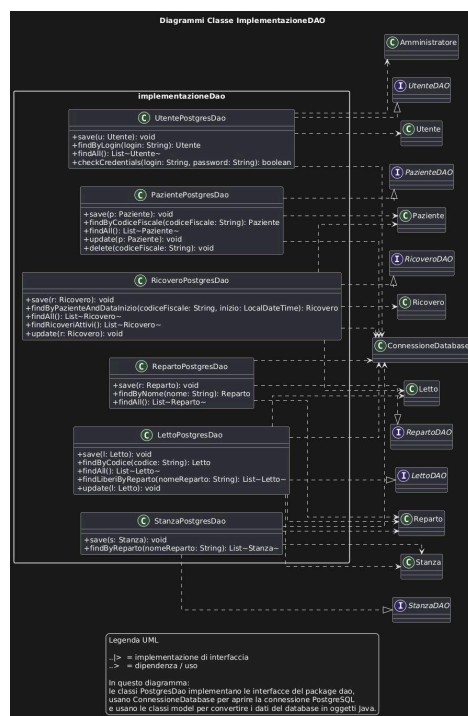
Diagrammi UML

Per mostrare l'architettura e l'organizzazione del codice abbiamo realizzato i diagrammi delle classi UML. Per evitare di avere uno schema unico e confusionario, abbiamo diviso la struttura in quattro diagrammi distinti, basati sui package principali:

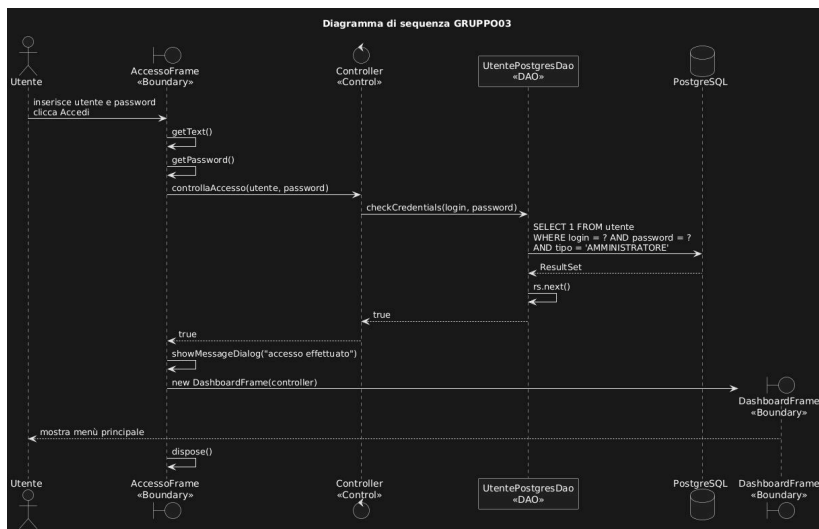
- **Package Model:** Descrive le classi base del nostro gestionale ospedaliero. Il diagramma mostra come sono strutturati i dati (tramite attributi, costruttori e metodi getter/setter) e fa capire come le varie classi sono collegate tra loro, evidenziando le gerarchie di ereditarietà e le composizioni.



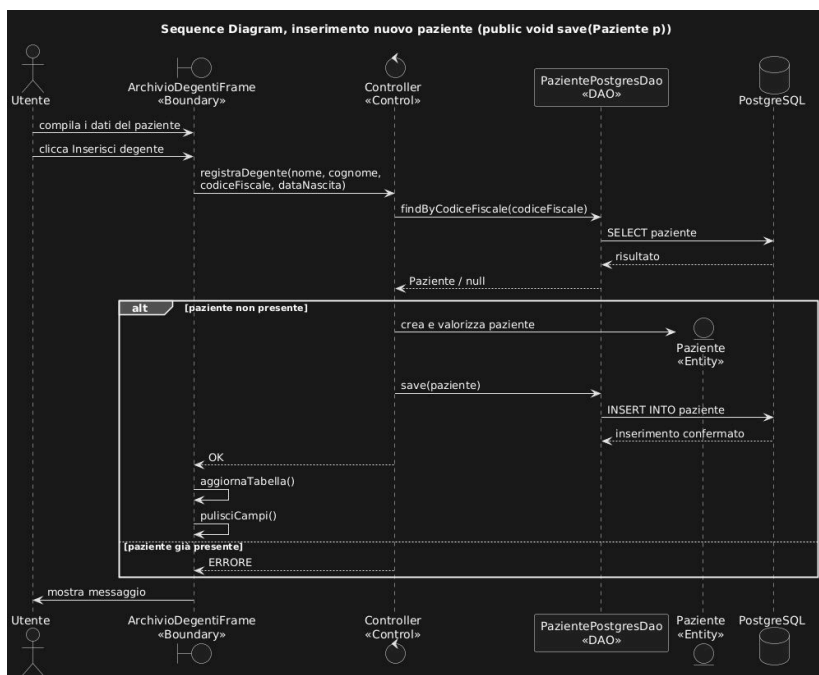
- **Package Implementazione DAO:** Contiene le classi vere e proprie che mettono in pratica i metodi delle interfacce. Il diagramma mostra le classi in cui abbiamo effettivamente scritto le query SQL necessarie per far dialogare l'applicazione con PostgreSQL.



Sequence Diagram



Il sequence diagram mostra il processo di login: l'utente inserisce le credenziali, AccessoFrame le invia al Controller, il Controller chiede a UtentePostgresDao di verificarle nel database PostgreSQL. Se le credenziali sono corrette, viene aperta la DashboardFrame e la finestra di login viene chiusa.



Il sequence diagram descrive l'inserimento di un nuovo paziente, dalla compilazione dei dati nella GUI fino al salvataggio nel database PostgreSQL tramite Controller e PazientePostgresDao.

Conclusioni

Il sistema, come previsto dalla consegna, permette di effettuare il login, visualizzare la dashboard, inserire nuovi pazienti e registrare ricoveri. I dati vengono salvati in un database PostgreSQL.

Il progetto ci ha permesso di applicare concretamente gli argomenti studiati durante il corso, come la programmazione a oggetti tramite Java, i diagrammi UML, il collegamento al database e soprattutto la gestione del progetto tramite sistemi di versioning come GitHub.